

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

TECHNICAL REPORT

Distributed Sensor Monitoring with HTTP, MQTT, SNMP, and SQLite

AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Embedded Systems

Contents

1 Mongoose 3

2 Distributed databases 3

3 APIs 3

4 LANs 3

5 Distributed systems 3

6 Virtual-machine topology 3

7 SQLite schema and initialization 4

8 Slave service 4

9 Master service and lookup path 4

10 Build and test automation 7

11 Security analysis 7

12 Two-layer architecture 7

13 Internal timing and the two-round benchmark 7

14 Protocol decisions and message path 11

15 Separating internal service time from total response time 15

16 Installation and agent configuration 15

17 OID structure 15

18 Request-to-response flow 16

19 Route and query contract 16

20 Submitted tests 18

21 Background service and monitoring loop 18

22 Trigger rules 18

23 Persistent alert schema 20

24 Delivery and scope **20**

1 Mongoose

Mongoose is a lightweight networking library for C and C++ that supports several protocols and is useful for embedded and IoT web services. A basic HTTP application includes mongoose.c and mongoose.h, initializes an event manager with mg_mgr_init, registers a listener with mg_http_listen, provides a request-event handler, and repeatedly calls mg_mgr_poll. This event-oriented organization keeps protocol handling within one application and is used by the sensor services in the following phases.

2 Distributed databases

Distributed databases store or manage data across nodes or locations, supporting different goals for capacity, availability, and resilience. The recorded examples are Apache Cassandra for large-scale write-oriented workloads, MongoDB for document storage and sharding, and Redis for rapid in-memory key/value access and caching. These products have different consistency and failure models; merely using separate SQLite databases does not reproduce their distributed-database guarantees.

3 APIs

An application programming interface specifies how programs request and use another component's services. It hides internal implementation behind a defined contract. In this project, a caller sends an HTTP request to a route and receives structured data, commonly JSON. A contract must identify accepted parameters, response fields, and errors, not just a URL.

4 LANs

A local-area network connects devices within a limited geographic area. Ethernet switches or Wi-Fi access points connect the nodes; TCP/IP supports application communication, and addresses identify endpoints. Such networks are commonly used to share Internet access, files, and equipment with relatively high bandwidth and low latency. The project uses a virtual LAN among Ubuntu nodes, so reachability, address configuration, and port access must be established before application tests.

5 Distributed systems

A distributed system comprises independent computers cooperating over a network. Processes run concurrently, no perfectly shared global clock is assumed, and failures may affect components independently. Distribution can enable capacity growth and fault isolation, but availability is a design outcome rather than an automatic property. Challenges include coordination, data consistency, network delay, partial failure, debugging, and securing communications. Those challenges motivate the explicit master/slave routing, bounded cache state, protocol adapters, and alert logic in this submission.

6 Virtual-machine topology

Three VMware virtual machines running Ubuntu 22.04 were created: master 192.168.64.128, slave 1 at 192.168.64.129, and slave 2 at 192.168.64.130. Their network adapters use NAT and obtain

distinct valid addresses from the local DHCP service. A ping from master to the slaves verifies basic reachability. These addresses document the experiment, not fixed settings that a reproduction must reuse. The applications accept communication parameters at runtime. NAT connectivity in this laboratory does not demonstrate externally reachable deployment or access restrictions for the slave ports.

7 SQLite schema and initialization

Each node maintains a local SQLite database with three principal tables. node_info stores node identity, master/slave role, and description. sensors stores sensor identifier, type/name, location, and unit. sensor_readings stores timestamped measurements and refers to sensors through a foreign key. Initialization scripts such as master_init_db.sh create tables and import the supplied CSV data into sensors and sensor_readings. This allows input files to change without editing the application. Indexes cover frequently used sensor identity/type and reading-time fields. read_latest_sample.sh joins the relevant tables and orders records by descending timestamp to obtain the newest measurement. A complete latest-value contract should specify handling of equal timestamps and invalid or absent data.

8 Slave service

The slave program is implemented in C++ with Mongoose. It listens for GET requests on /api/sensor. The listening port and database path are command-line inputs. It extracts type and id from the URL, queries the local SQLite database for the latest reading, and formats a JSON response. A local curl request and Mongoose log demonstrate operation on port 8081 after compilation with g++ and linkage to SQLite.

9 Master service and lookup path

The master is also a C++ Mongoose service. It is the operator-facing entry point and has its own local database. It first searches locally and labels a local response with node: Master. On a miss, it sends an HTTP request to slave 1; following a not-found result, it tries slave 2. It returns the successful slave result or a final JSON error if all three nodes lack the requested reading. The supplied implementation is master-orchestrated fallback. The experiment report calls it cascading, but slave 1 does not itself forward the request to slave 2. This distinction matters for timeout behavior, master load, and conformance to the specified forwarding path. The master receives its port, database location, and slave endpoint settings through command-line arguments. Dynamic configuration facilitates relocation; it does not itself guarantee unlimited scalability. For example, the operator requests /api/sensor?type=smoke&id=304. A local hit returns HTTP 200. Otherwise the master requests slave 1, then slave 2 if appropriate. If none has the reading, it returns HTTP 404 with a defined error. A robust gateway should distinguish absent data from timeout or an unavailable node rather than map every failure to not found.

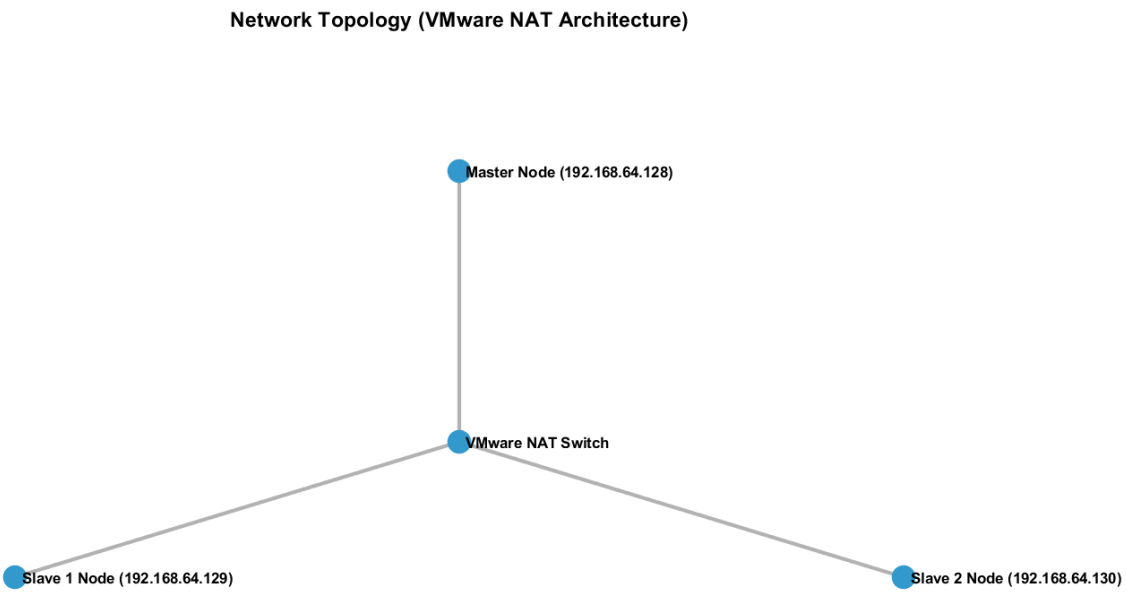


Figure 1: Diagram or recorded result: Master service and lookup path.

```
reza@master:~/for_students/scripts$ ./read_latest_sample.sh master.db temperature 101
./read_latest_sample.sh slave1.db co2 204
./read_latest_sample.sh slave2.db smoke 304
sensor_id  sensor_type  sensor_name          location        value  unit  recorded_at
---------  -----------  -------------------  --------------  -----  ----  --------------
101        temperature  Floor1_Room101_Temp  Floor1_Room101  24.8   C     6/1/2026 10:15
sensor_id  sensor_type  sensor_name          location            value  unit  recorded_at
---------  -----------  -------------------  ------------------  -----  ----  --------------
204        co2          Floor2_Meeting_CO2   Floor2_MeetingRoom  735    ppm   6/1/2026 10:20
sensor_id  sensor_type  sensor_name           location        value  unit  recorded_at
---------  -----------  --------------------  --------------  -----  ----  --------------
304        smoke        Floor3_Storage_Smoke  Floor3_Storage  0      bool  6/1/2026 10:20
```

Figure 2: Diagram or recorded result: Master service and lookup path.

```
reza@slave1: ~/CHW4/Q1/sensor_network
reza@slave1:~/CHW4/Q1/sensor_network$ g++ main.cpp mongoose.c -o slave_node -lsqlite3
reza@slave1:~/CHW4/Q1/sensor_network$ ./slave_node 8081 ~/slave1.db
29a9f4 3 mongoose.c:7806:              MG_IO_SIZE: 16384, TLS: none
29a9f9 3 mongoose.c:7719:              1 4 http://0.0.0.0:8081
Slave Node starting on http://0.0.0.0:8081 using DB: /home/reza/slave1.db
2a14e3 3 mongoose.c:14239:             2 5 accepted 192.168.64.128:46028 -> 192.168.64.129:8081
2a14e3 3 mongoose.c:14082:             2 5 0:0:0 109 err 0
2a14e4 3 mongoose.c:14092:             2 5 snd 130/16384 rcv 0/16384 n=130 err=0
2a14e5 3 mongoose.c:14082:             2 5 0:0:0 -1 err 2
2a14e5 3 mongoose.c:7655:              2 5 closed
```

Figure 3: Diagram or recorded result: Master service and lookup path.

```
reza@master:~/for_students/scripts$ curl "http://192.168.64.129:8081/api/sensor?type=co2&id=204"
{"status": "success", "value": 735, "unit": "ppm"}reza@master:~/for_students/scripts$
```

Figure 4: Diagram or recorded result: Master service and lookup path.

```
reza@master:~/sensor_network/master$ ./master_node 8080 ./master.db 192.168.64.129:8081 192.168.64.130:8082
3751bf 3 mongoose.c:7806:               MG_IO_SIZE: 16384, TLS: none
3751c0 3 mongoose.c:7719:               1 4 http://0.0.0.0:8080
Master Node starting on http://0.0.0.0:8080
Local DB: ./master.db
Slave 1: http://192.168.64.129:8081
Slave 2: http://192.168.64.130:8082
377bbc 3 mongoose.c:14239:              2 5 accepted 127.0.0.1:48552 -> 127.0.0.1:8080
377bbc 3 mongoose.c:14082:              2 5 0:0:0 112 err 0
377bbd 3 mongoose.c:14092:              2 5 snd 147/16384 rcv 0/16384 n=147 err=0
377bbd 3 mongoose.c:14082:              2 5 0:0:0 -1 err 2
377bbd 3 mongoose.c:7655:               2 5 closed
37ca76 3 mongoose.c:14239:              3 5 accepted 127.0.0.1:54128 -> 127.0.0.1:8080
37ca76 3 mongoose.c:14082:              3 5 0:0:0 104 err 0
Data not in Master, asking Slave 1...
37ca7f 3 mongoose.c:14092:              3 5 snd 148/16384 rcv 0/16384 n=148 err=0
37ca7f 3 mongoose.c:14082:              3 5 0:0:0 -1 err 2
37ca7f 3 mongoose.c:7655:               3 5 closed
```

Figure 5: Diagram or recorded result: Master service and lookup path.

```
reza@master:~/sensor_network/master$ curl "http://localhost:8080/api/sensor?type=temperature&id=101"
{"status": "success", "value": 24.8, "unit": "C", "node": "Master"}reza@master:~/sensor_network/master$ curl "http://localhost:8080/api/sensor?type=tempe
co2&id=204"
{"status": "success", "value": 735, "unit": "ppm", "node": "Slave1"}reza@master:~/sensor_network/master$
```

Figure 6: Diagram or recorded result: Master service and lookup path.

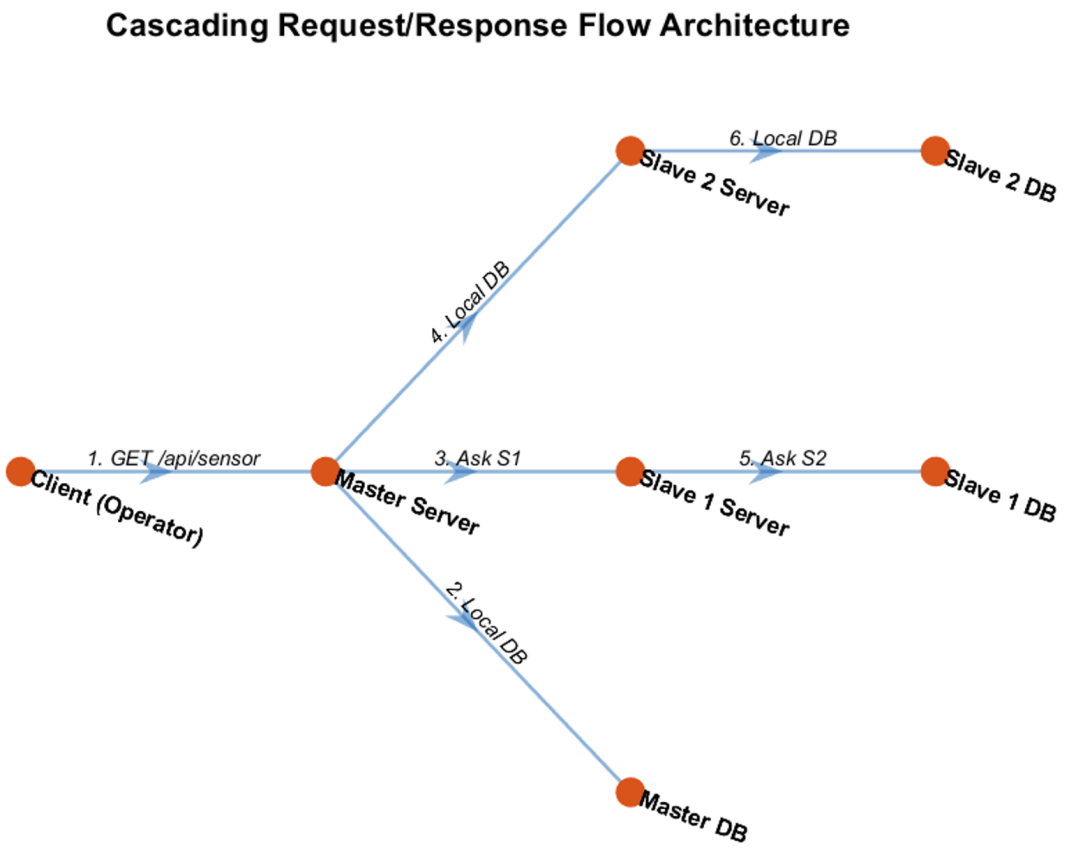


Figure 7: Diagram or recorded result: Master service and lookup path.

10 Build and test automation

Separate Makefiles compile master and slave sources and link Mongoose and SQLite. scripts/build_and_run.sh clears the previous build, compiles the applications, and displays the master invocation with its arguments. The code directory contains the scripts, Makefiles, and execution instructions for each stage.

11 Security analysis

The demonstrated operator/master and master/slave communication uses plain HTTP. Readings can be observed or modified by an attacker positioned on the network. No application authentication is demonstrated. Although the intended operator path is through the master, exposed slave ports can still accept direct requests unless a firewall prevents them. Further security measures include TLS/HTTPS, API-key or bearer-token authentication, firewall rules restricting slave requests to the master, and safe parameterized SQL handling. These remain proposals unless implemented in the source. The report recordedly attributes complete SQL-injection protection to sqlite3_prepare_v2 and sqlite3_bind_text; source inspection shows that preparing a concatenated query does not itself provide parameter binding. External values must be bound to placeholders and validated. No single API call justifies a complete-injection-immunity claim.

12 Two-layer architecture

Memcached is added to the SQLite-backed nodes. A key identifies a sensor, for example temperature_101. On a request, the service first checks the key. A hit returns the cached value. A miss queries local SQLite or obtains the result from the appropriate downstream node, then stores the successful response with a 60-second TTL before returning it. The main database retains durable readings; the cache is temporary and does not replace persistent storage. Expiration can explain a second-round miss if more than 60 seconds pass. Memory pressure and eviction can also remove a key. A current-reading API must consider stale data: an insertion or update should invalidate or refresh the matching key, otherwise a reading can remain old until its TTL expires. The experiment report's hit/miss path and figures are preserved; a fully consistent distributed invalidation policy is not established.

13 Internal timing and the two-round benchmark

The application measures response processing using the C++ chrono facilities. A first-round miss performs database work and possibly an HTTP lookup. A second-round hit obtains the retained result from RAM. The report includes an example described as more than thirteen times faster, and a separate script output with the following observed times. They should be kept as separate observations rather than combined into a single universal speedup.

Read path	First round (ms)	Second round (ms)	Ratio for these observations
Local reading	2.4	0.08	30
Remote example 1	21.7	0.2	108.5
Remote example 2	10.2	0.2	51

The supplied benchmark is launched from scripts after making benchmark_read.sh executable. It reads the sensors in two rounds and prints the timings. The large ratios show the benefit of serving

a retained response in this small experiment. They do not prove that network overhead disappears in general, that every request is a hit, or that the system remains equally fast at scale. Dataset size, cache capacity, workload distribution, expiry, and client concurrency must be reported in a broader evaluation.

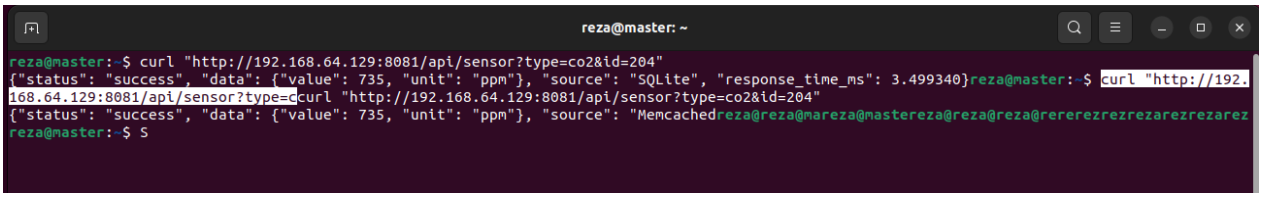


Figure 8: Diagram or recorded result: Internal timing and the two-round benchmark.

```
reza@master:~/sensor_network_p2/scripts$ ./benchmark_read.sh
================================================
   BENCHMARK SCRIPT - ROUND 1 (CACHE MISS)
================================================
Reading from Master Local DB:
{
  "status": "success",
  "data": {
    "value": 24.8,
    "unit": "C"
  },
  "source": "SQLite",
  "node": "Master",
  "response_time_ms": 2.401153
}

Reading from Slave 1 (via Master):
{
  "status": "success",
  "data": {
    "value": 735,
    "unit": "ppm"
  },
  "source": "Network",
  "node": "Slave1",
  "response_time_ms": 10.268724
}

Reading from Slave 2 (via Master):
{
  "status": "success",
  "data": {
    "value": 0,
    "unit": "bool"
  },
  "source": "Network",
  "node": "Slave2",
  "response_time_ms": 21.737109
}
```

Figure 9: Diagram or recorded result: Internal timing and the two-round benchmark.

```
==============================================
   BENCHMARK SCRIPT - ROUND 2 (CACHE HIT)
==============================================
Reading from Master Local DB:
{
  "status": "success",
  "data": {
    "value": 24.8,
    "unit": "C"
  },
  "source": "Master_Memcached",
  "node": "Master",
  "response_time_ms": 0.087785
}

Reading from Slave 1 (via Master):
{
  "status": "success",
  "data": {
    "value": 735,
    "unit": "ppm"
  },
  "source": "Master_Memcached",
  "node": "Master",
  "response_time_ms": 0.200453
}

Reading from Slave 2 (via Master):
{
  "status": "success",
  "data": {
    "value": 0,
    "unit": "bool"
  },
  "source": "Master_Memcached",
  "node": "Master",
  "response_time_ms": 0.199368
}
```

Figure 10: Diagram or recorded result: Internal timing and the two-round benchmark.

14 Protocol decisions and message path

The master extends its HTTP service with MQTT publish/subscribe request handling. The submission chooses MQTT 3.1.1 for its compact protocol and established support. The recorded argument is that simple sensor lookup does not require version-5 message properties. This is a project choice rather than a claim that version 5 is inappropriate for all embedded systems. QoS 1 is selected as a compromise between delivery acknowledgment and the additional exchange used by QoS 2. It supports at-least-once delivery for the relevant protocol exchange, so duplicate requests and responses should be tolerated. It does not automatically guarantee that an application action is executed once or that a response reaches an unavailable subscriber. The operator communicates through the broker. The master acts as a gateway and performs the cached/local/remote lookup; on a cache miss it can contact the slaves over HTTP.

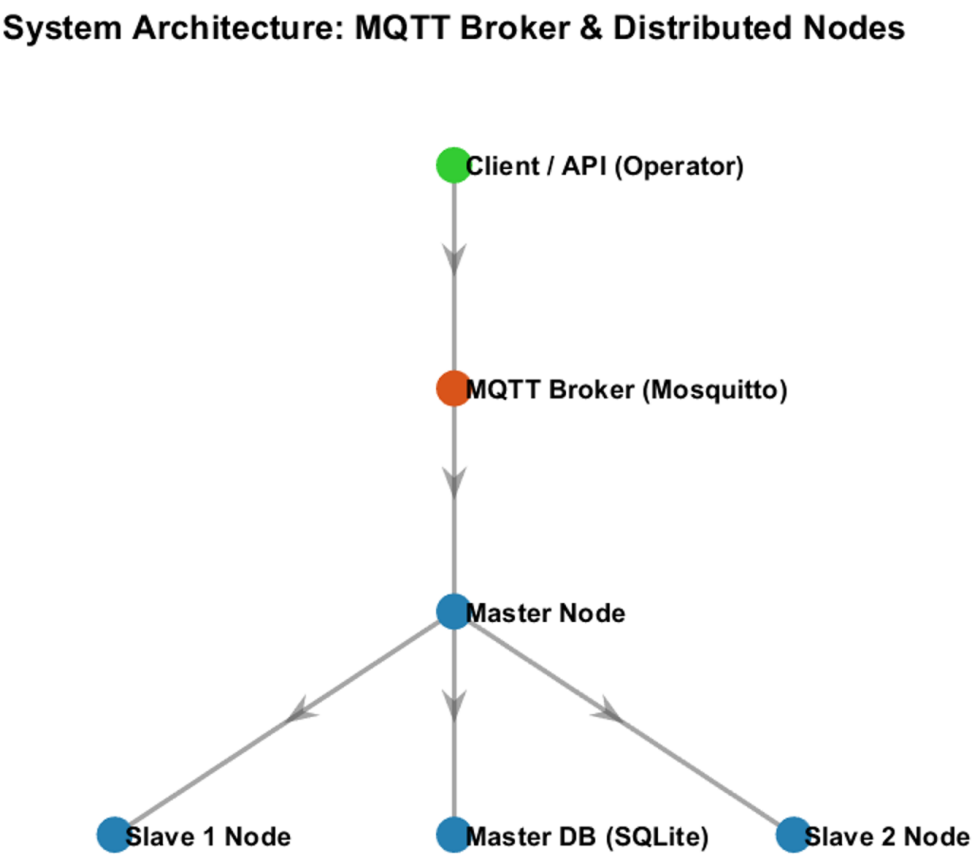


Figure 11: Diagram or recorded result: Protocol decisions and message path.

```
reza@master:~/sensor_network_p3/scripts$ ./mqtt_benchmark.sh
==========================================
 MQTT BENCHMARK SCRIPT - ROUND 1 (CACHE MISS)
==========================================
Reading from Master Local DB:
Response Time (MQTT): 52 ms
{
  "status": "success",
  "data": {
    "value": 24.8,
    "unit": "C"
  },
  "source": "SQLite",
  "node": "Master",
  "response_time_ms": 4.667523
}
----------------------------------------
Reading from Slave 1 (via Master):
Response Time (MQTT): 50 ms
{
  "status": "success",
  "data": {
    "value": 735,
    "unit": "ppm"
  },
  "source": "Network",
  "node": "Slave1",
  "response_time_ms": 44.009895
}
----------------------------------------
Reading from Slave 2 (via Master):
Response Time (MQTT): 48 ms
{
  "status": "success",
  "data": {
    "value": 0,
    "unit": "bool"
  },
  "source": "Network",
  "node": "Slave2",
  "response_time_ms": 28.973386
}
----------------------------------------
```

Figure 12: Diagram or recorded result: Protocol decisions and message path.

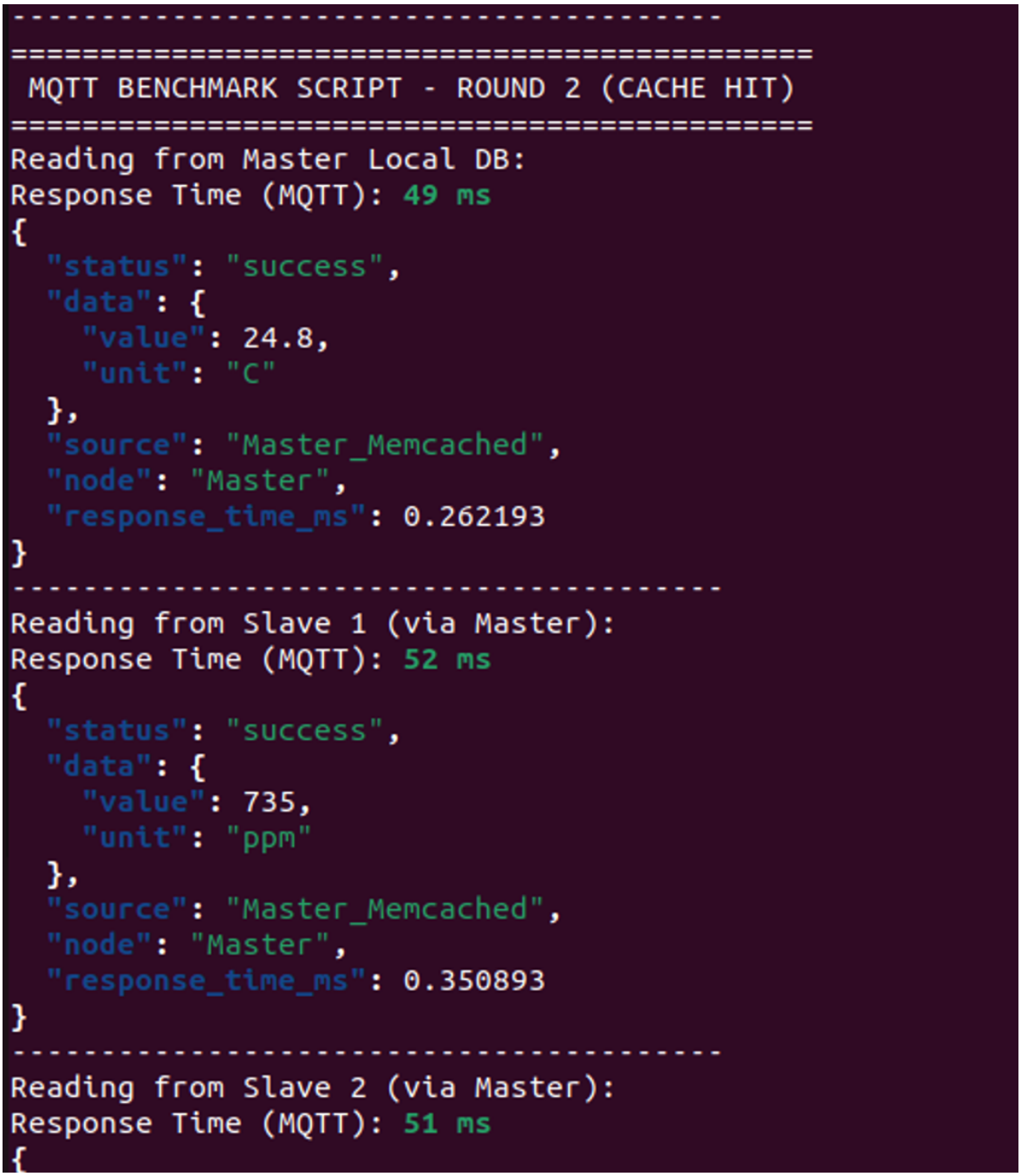


Figure 13: Diagram or recorded result: Protocol decisions and message path. Panel 1 of 2.

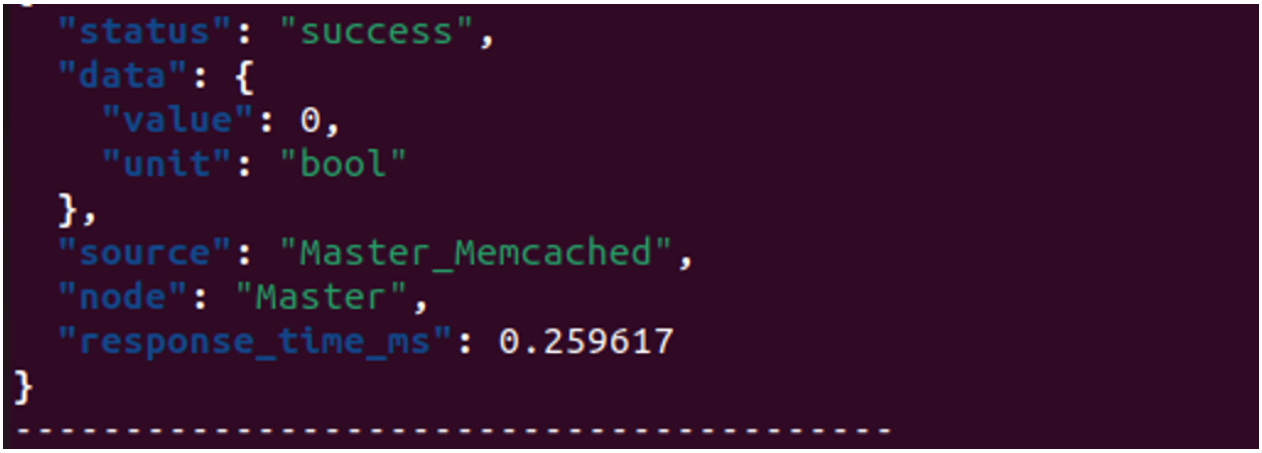


Figure 14: Diagram or recorded result: Protocol decisions and message path. Panel 2 of 2.

15 Separating internal service time from total response time

The output reports two different timings. response_time_ms measures processing within the master, including local SQLite or downstream lookup. First-round examples range from approximately 4.6 to 44 ms; second-round cached examples fall to approximately 0.2 ms. The client-observed Response Time includes connection/setup, publication, application handling, and response receipt, and remains approximately 48 to 52 ms in both rounds. The cache reduces the internal lookup work, but the reported end-to-end MQTT exchange remains near 50 ms. It is therefore incorrect to apply the internal cache speedup to the complete client path or to infer a bottleneck-free architecture from these few observations. Fixed client overhead, the script's timing method, and broker/client scheduling may dominate this workload. The experiment demonstrates protocol integration and different timing boundaries, not a verified large-scale performance result.

16 Installation and agent configuration

The master exposes sensor information through snmpd. The recorded installation commands update package metadata and install snmpd, snmp, and jq. A read-only community is configured for local requests. A pass directive binds the illustrative enterprise subtree .1.3.6.1.4.1.55555 to scripts/snmp_pass.sh through Bash. The agent is restarted with systemctl after configuration changes. The example subtree is a project-local choice; it is not documented as an allocated private enterprise number. SNMPv2c community strings do not provide encrypted, cryptographically authenticated monitoring. Restrict access appropriately in a reproduction and consider SNMPv3 for a deployment requiring those properties.

17 OID structure

Each demonstrated sensor has entries for name, type/description, and latest value. The master temperature sensor 101 uses suffixes .1.1, .1.2, and .1.3 under the chosen root. Slave-1 CO2 sensor 204 uses .2.1 through .2.3. Slave-2 smoke sensor 304 uses .3.1 through .3.3. This organization is retained from the experiment report and configuration.

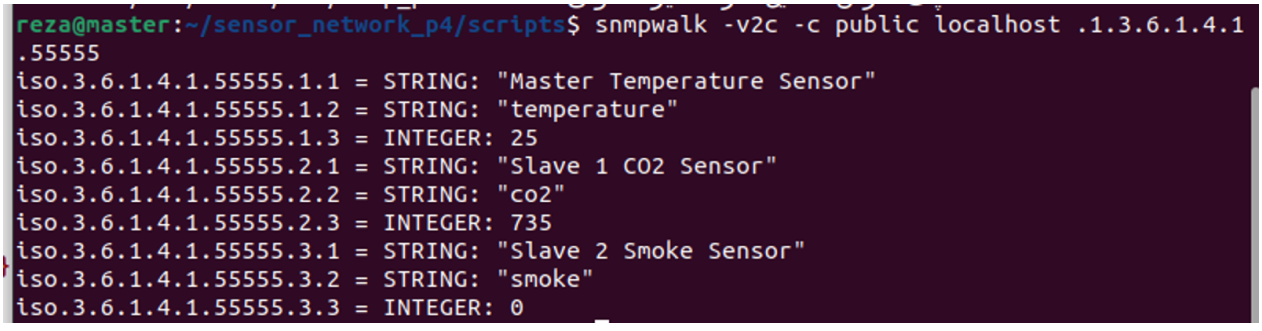


Figure 15: Diagram or recorded result: OID structure.

18 Request-to-response flow

The operator walks the subtree with snmpwalk using version 2c. snmpd invokes the pass script with the requested OID. The script sends an HTTP request with curl to the master API on its configured port, rather than directly reading an unrelated database. The master then retrieves the reading through the existing cache/local/remote path. The script prints the result in the format expected by the SNMP adapter and snmpd returns it to the client. The all-sensor test is run by making read_snmp.sh executable and launching it from scripts. This architecture reuses the HTTP lookup contract, but its performance and availability also depend on that service; the existence of an SNMP adapter does not eliminate the master as a central dependency.

19 Route and query contract

The Mongoose service provides /api/sensor/logs. It accepts sensor_name, sensor_id, and date in YYYY-MM-DD format. A request first reaches the master, which searches its local database. If no matching history is present, it forwards the request to slave services and returns their structured result. Local and remote database rows are not fabricated from fixed examples. The report illustrates selecting time(recorded_at) and value from sensor_readings for a specified calendar date. A complete implementation also identifies the requested sensor and uses parameter binding for caller-supplied inputs. Date format, missing parameters, empty histories, invalid identifiers, and network failures need distinguishable behavior.

```
reza@master:~/sensor_network_p4/master$ curl -s "http://localhost:8080/api/sensor/logs?sensor_name=temperature
&sensor_id=101&date=2026-06-01" | jq .
{
  "sensor_name": "temperature",
  "sensor_id": "101",
  "date": "2026-06-01",
  "values": [
    {
      "time": "10:15:00",
      "value": "24.8"
    },
    {
      "time": "10:30:00",
      "value": "25.1"
    },
    {
      "time": "10:45:00",
      "value": "25.4"
    }
  ]
}
```

Figure 16: Diagram or recorded result: Route and query contract.

```
reza@master:~/sensor_network_p4/master$ curl -s "http://localhost:8080/api/sensor/logs?sensor_name=co2&sensor_
id=204&date=2026-06-01" | jq .
{
  "sensor_name": "co2",
  "sensor_id": "204",
  "date": "2026-06-01",
  "values": [
    {
      "time": "11:00:00",
      "value": "700"
    },
    {
      "time": "11:15:00",
      "value": "715"
    }
  ]
}
```

Figure 17: Diagram or recorded result: Route and query contract.



Figure 18: Diagram or recorded result: Route and query contract.

20 Submitted tests

The first test requests the master's temperature-sensor history on 2026-06-01. The second requests CO2 history located on slave 1 through the master. The third requests an absent date, 2020-01-01, and receives a defined no-data response. These cases demonstrate local lookup, remote lookup, and an empty-result path; they do not establish exhaustive validation or all network-error behavior. Returned JSON identifies the sensor/date and contains an array of time/value entries. Timezone and ordering should be explicit in a production contract, because interpreting a calendar day through SQLite date functions can differ from an operator's intended local day.

21 Background service and monitoring loop

The alert process runs independently under systemd. Every five seconds it retrieves the newest available record for each sensor and compares its time/value with the configured rules. SQLite is configured in WAL mode to improve overlap between reading and writing. WAL reduces some contention but does not guarantee an absence of locks, unlimited concurrent writers, or immunity to busy errors. Before inserting an alert, the program checks for a matching outstanding event with status NEW. If one exists, it avoids inserting the same active condition again. This bounds repeated alerts while preserving a persistent record that requires acknowledgment. A complete lifecycle would define transitions such as acknowledgment and resolution; they are not inferred from the duplicate-suppression mechanism alone.

22 Trigger rules

Alert type	**Recorded rule**
TIMEOUT_NO_DATA	Latest known reading is older than 300 seconds
INVALID_DATA	Reading below -50 or above 1000
HIGH_TEMPERATURE	Temperature above 40 C
HUMIDITY_OUT_OF_BOUNDS	Humidity below 20 percent or above 80 percent

These thresholds document the submitted configuration, not universal limits for every sensor. In particular, units and admissible ranges must be associated with sensor type. A query based only on existing readings cannot detect an expected sensor that has never produced a row; detecting that case requires joining the sensor registry to its readings.

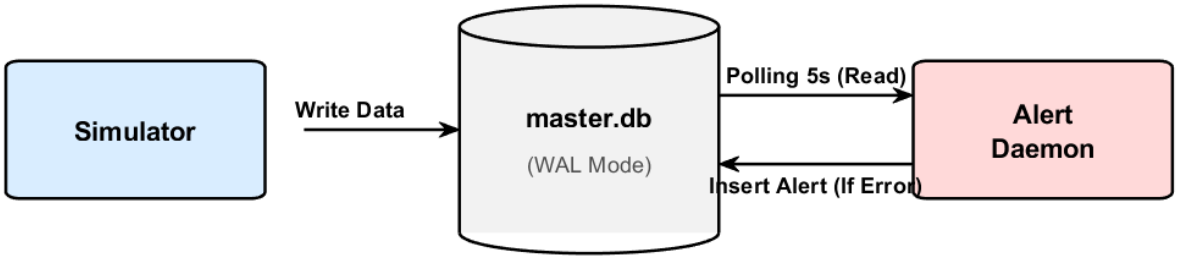


Figure 19: Alert-service architecture: Trigger rules.

```
reza@master:~/sensor_network_p6/master$ journalctl -u sensor-alert -f
15:34:50 12 July   master systemd[1]: sensor-alert.service: Deactivated successfully.
15:34:50 12 July   master systemd[1]: Stopped IoT Sensor Alert Daemon.
15:34:50 12 July   master systemd[1]: Started IoT Sensor Alert Daemon.
15:34:50 12 July   master alert_daemon[6438]: IoT Alert Daemon Started Successfully.
15:41:04 12 July   master systemd[1]: Stopping IoT Sensor Alert Daemon...
15:41:04 12 July   master systemd[1]: sensor-alert.service: Deactivated successfully.
15:41:04 12 July   master systemd[1]: Stopped IoT Sensor Alert Daemon.
15:41:04 12 July   master systemd[1]: Started IoT Sensor Alert Daemon.
15:41:04 12 July   master alert_daemon[6616]: IoT Alert Daemon Started Successfully.
15:41:04 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:43:59 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:44:19 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:44:24 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:44:29 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:44:34 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:48:59 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:49:24 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.000000
15:49:34 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 101 (temperature) | HIGH_TEMPERATURE | Value: 99.900000
15:49:34 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 102 (humidity) | INVALID_DATA | Value: -100.000000
15:49:34 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 102 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: -100.000000
```

Figure 20: Diagram or recorded result: Trigger rules.

```
reza@master:~/sensor_network_p6/scripts$ sqlite3 ~/sensor_network_p6/master/master.db "SELECT * FROM alerts;"
8|103|humidity|HUMIDITY_OUT_OF_BOUNDS|1.000000|2026-07-12 12:19:24|NEW
9|101|temperature|HIGH_TEMPERATURE|99.900000|2026-07-12 12:19:34|NEW
10|102|humidity|INVALID_DATA|-100.000000|2026-07-12 12:19:34|NEW
11|102|humidity|HUMIDITY_OUT_OF_BOUNDS|-100.000000|2026-07-12 12:19:34|NEW
reza@master:~/sensor_network_p6/scripts$
```

Figure 21: Diagram or recorded result: Trigger rules.

```
reza@master:~/sensor_network_p6/scripts$ sudo systemctl status sensor-alert
[sudo] password for reza:
● sensor-alert.service - IoT Sensor Alert Daemon
     Loaded: loaded (/etc/systemd/system/sensor-alert.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2026-07-12 15:41:04 +0330; 17min ago
   Main PID: 6616 (alert_daemon)
      Tasks: 1 (limit: 6960)
     Memory: 528.0K
        CPU: 284ms
     CGroup: /system.slice/sensor-alert.service
             └─6616 /home/reza/sensor_network_p6/master/alert_daemon

15:43:59 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.0
15:44:19 12 July   master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.0
15:44:24 12 جولای master alert_daemon[6616]: [ALERT LOGGED] Sensor 103 (humidity) | HUMIDITY_OUT_OF_BOUNDS | Value: 1.0
```

Figure 22: Implementation and output: Trigger rules.

23 Persistent alert schema

The alerts table has an increasing integer primary key id; sensor_id; sensor_name; alert_type; sensor_value; a created_at timestamp; and status, initially NEW. It records the value observed when the rule is triggered and gives an operator a durable list to inspect.

24 Delivery and scope

The source is implemented in C++. The submission includes a combined master build script, a service-installation/start script, and a database initializer that creates the alert table and enables WAL. The daemon monitors the configured local master database. The evidence does not demonstrate automatic monitoring of every slave database, every expected sensor, or complete node-failure coverage. The English report distinguishes this implemented scope from the broader monitoring objective instead of claiming a distributed alert capability that has not been established.